ENTERPRISE LLM ARCHITECTURE

Study Guide — Tokens, Prompt Architecture, Grounding & Enterprise Safety

Week 2 · Day 1 · Prepared by Rajesh Pasham

Program Context

Item	Detail
Session Type	Concepts and architecture lecture — no live Palantir platform work today
Platform	Individual Palantir Developer Tier accounts, used from Day 3 onward
This Week's Lab (Day 3+)	Build a Copilot grounded in your batch's Week 1 Ontology — each batch's own use case

Learning Objectives

By the end of today, you will be able to:

- Explain what changes about LLM architecture at enterprise scale: tokens, context windows, and model limitations
- Apply model-selection criteria deliberately, rather than defaulting to one model for every task
- Structure a prompt using system, user, and contextual instruction layers
- Choose between zero-shot and few-shot technique, and request structured output where a system needs to consume the response
- Ground an LLM response in Ontology data, and explain how that differs from and complements standard RAG
- Reason about model routing, hallucination, ambiguity, and enterprise security as architecture decisions

1. Why Enterprise Architecture Is Different

It isn't the model itself that makes an architecture “enterprise” — it's everything built around it. A single governed Ontology and security model has to serve every Copilot built on top of it, not just one prototype. Every answer has to be accountable: who asked, what was retrieved, what was said. And cost and latency are architecture decisions made deliberately, not afterthoughts discovered once something is already in production.

2. Tokens & Context Windows

Term	Definition
Token	The unit a model reads and generates in — roughly a word-fragment, not a full word or a single character
Context Window	The maximum number of tokens a model can consider at once — system prompt, conversation history, and retrieved context all share this same budget

Retrieving “more context” is not free. It competes for space in the same window as everything else, and can push out what actually matters to answering the question well.

3. Model Limitations to Design Around

- Knowledge cutoffs — a model's training data has a boundary in time, and it won't reliably tell you when it's answering from outside that boundary
- Latency under real load — a response that's fast in a demo can behave differently under concurrent enterprise traffic
- Consistency, not certainty — the same prompt can produce different phrasing across runs; architecture should tolerate that rather than assume determinism

4. Model-Selection Criteria

Criterion	What to Ask
Task Complexity	Does this task actually need the largest, most capable model available, or would a smaller one do it reliably?
Latency Requirements	Is this interactive (needs a fast response) or batch (can tolerate longer processing)?
Data Sensitivity	Is a Palantir-hosted model sufficient, or does what the model will see require a registered/bring-your-own model?
Cost at Scale	What does the per-call cost look like at production call volume, not just in testing?

5. Three Layers of Instruction

Layer	Answers
System	Who is this application, and what are its permanent rules? Set once, rarely changes per call.
User	What is this specific person asking right now?
Contextual	What retrieved data or application state is relevant to answering it?

System instructions are designed once; user and contextual instructions are designed to vary on every call. Confusing the two — for example, hardcoding per-call data into what should be a permanent system rule — is a common source of inconsistent Copilot behavior.

6. System Instructions in Practice

- Lead with the overview — what is this Copilot for, in one or two sentences, before any other detail
- State constraints explicitly, not implicitly — “summarize and flag” is a different instruction from “recommend an approval,” and the difference should be spelled out, not assumed obvious
- This is where “never do X” belongs — stated once here, it applies to every user interaction, rather than needing to be re-stated per question

7. Zero-Shot vs. Few-Shot Techniques

Technique	Description
Zero-Shot	No examples included in the prompt, just instructions. Cheapest in tokens; relies entirely on the model's general capability.
Few-Shot	A small number of worked examples included in the prompt. Improves consistency for tricky formats, at a real token cost.

A reasonable default: start zero-shot. Add examples only once you've observed a specific, repeatable failure mode that examples would actually fix — not preemptively, just in case.

8. Structured Output

Free text is for people; structured output is for systems. If a downstream Function or Action needs to consume the model's answer, request a defined shape — specific field names and types — rather than prose that has to be parsed afterward. Even when structured output is requested, a model can still return malformed results; downstream validation remains your responsibility, not an optional extra step.

9. Prompt Templates

The same five-part structure — Overview, Context, Tools, Constraints, Output — should be reusable across every Copilot your batch builds, not reinvented each time. What stays fixed is the section structure and the constraints pattern; what varies is the specific data, tools, and domain language for each use case — your batch's Claims, Underwriting, Fraud, or Servicing specifics.

10. Context Engineering at Enterprise Scale

- Context is retrieved, not pasted — what the model sees should come from a governed retrieval step, not a static block of text someone wrote once
- Governance travels with the data — if a Week 1 Marking protects a field, that protection has to hold when an LLM retrieves it too, not only when a person views it directly
- Less, well-chosen context beats more, unfiltered context — a shorter, relevant context window produces better answers than a longer, noisy one

11. Grounding LLMs with Ontology Data

Grounding means the model's answer is built from retrieved, governed Ontology objects — your batch's Claim, Application, Fraud Case, or Policy records — rather than from what the model generally “remembers” about insurance from its training data.

12. Retrieval-Augmented Generation (RAG)

The core idea behind RAG: retrieve relevant data first, then include it in the prompt, rather than relying on the model's training data alone. Standard RAG typically retrieves unstructured text chunks from documents. Ontology-grounded retrieval instead retrieves governed, structured Ontology objects, anchored in your Week 1 model rather than just similar-sounding text. Most production Copilots use both approaches together.

13. Model-Routing Patterns

- Route by task complexity — a simple lookup doesn't need the same model as a multi-step reasoning task; route accordingly to control cost
- Route by data sensitivity — a question touching highly sensitive fields may need to route to a registered model rather than a default Palantir-hosted one
- Route by confidence — a low-confidence response can route to a different model, or to a human, rather than being returned as-is

14. Handling Hallucination

Hallucination is a model producing a fluent, confident answer that isn't actually supported by the retrieved data. The architectural response is to ground every claim in retrieved context and require the model to cite what it retrieved — not simply to ask it to "be accurate." No prompt eliminates hallucination completely; this is exactly why Week 6's Evals exist — to measure it, rather than assume it away.

15. Handling Ambiguity

Ambiguity is not always a bug to fix. Some questions genuinely have more than one reasonable interpretation, and a well-designed Copilot should say so rather than silently pick one. This is why an explicit escalation path matters: a Copilot that isn't confident should say so and route to a human — exactly the escalation pattern specified in this week's Practical Lab.

16. Enterprise Security Considerations

- Roles and Markings still gate what's retrieved — an LLM querying the Ontology is still a query, and your Week 1 governance model applies to it exactly as it applies to a person
- Prompt injection is a real, specific risk — retrieved content (a document, a user question) can contain text attempting to override system instructions; design defensively rather than assuming retrieved content is inert
- Every model call should be auditable — which model, what was retrieved, what was returned — traceable, not a black box

17. Common Pitfalls

- Picking a model before defining the task — model choice should follow task complexity, latency needs, and data sensitivity, not the other way around
- Mixing system and contextual instructions — permanent rules and per-call data belong in different layers; blurring them produces inconsistent behavior
- Retrieving more context "to be safe" — more context competes for the same window as everything else; targeted retrieval beats broad retrieval
- Treating hallucination as a prompt-wording problem — it's an architecture problem; grounding, citation, and Evals address it, clever wording alone doesn't

18. Glossary

Term	Meaning
Token	The unit a model reads and generates in, roughly a word-fragment
Context Window	The maximum number of tokens a model can consider at once
System / User / Contextual Instructions	The three layers a compiled prompt is assembled from
Zero-Shot / Few-Shot	Prompting with no examples vs. a small number of worked examples
Structured Output	A model response constrained to a defined, machine-consumable format
Grounding	Building a model's answer from retrieved, governed data rather than general training knowledge
RAG	Retrieval-Augmented Generation — retrieving relevant data before generating a response
Model Routing	Directing a request to a specific model based on complexity, sensitivity, or confidence
Hallucination	A fluent, confident model response not actually supported by retrieved data

19. Further Practice

Continue into Day 2, where these architectural concepts extend into LLM pipeline design and the AI Forward Deployed Engineering workflow — the bridge into Day 3's hands-on Copilot build.